



Linear Models

- Linear Classifier
- Perceptron
- Logistic Regression
- Example

Linear Classifiers



Discriminant function as a **linear combination** of features

$$f(x) = \beta^T x + \beta_0$$

- Quite **simple**: one of the first algorithms to try
- With **very large datasets** and **complex tasks** it could be the only available options (being simple, it has a low cost)

Perceptron



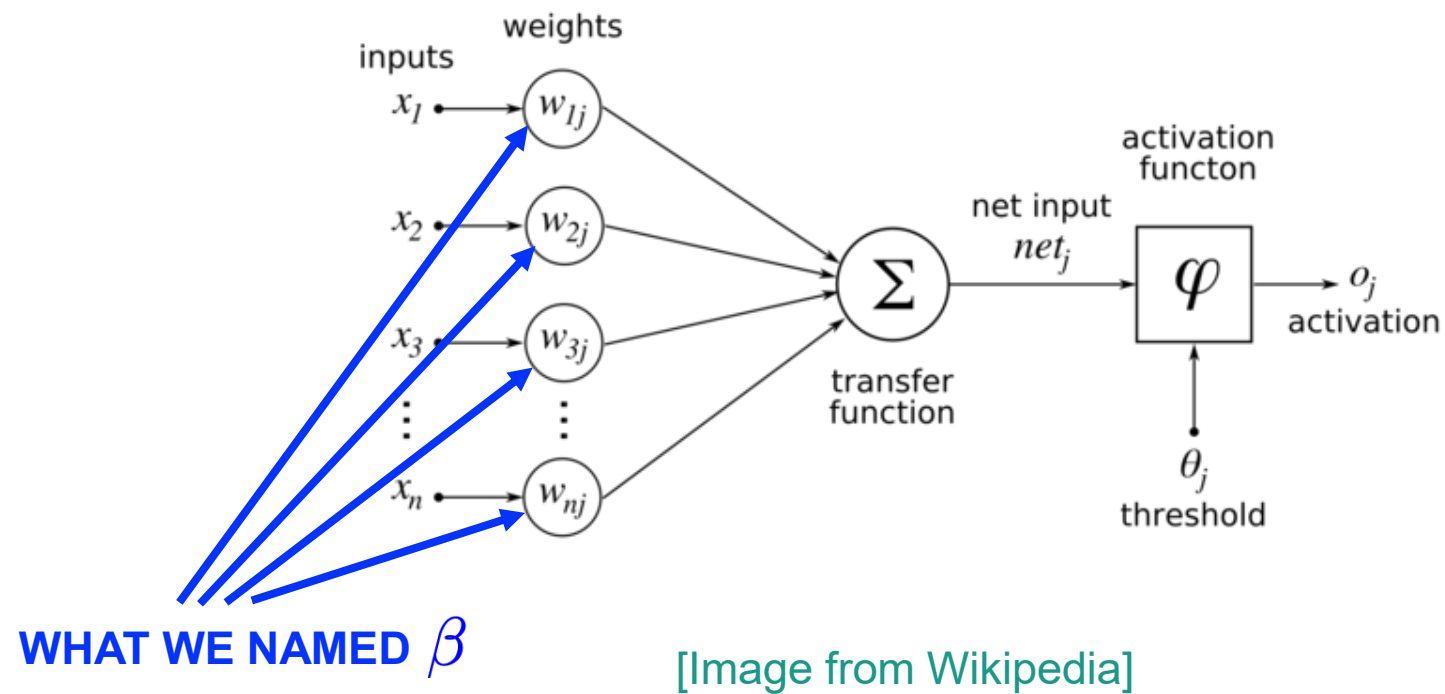
Architecture with a **single neuron**

$$f(x) = \text{sign}(\beta^T x + \beta_0)$$

- Biological inspiration: **synopsis activation** when the neuron potential exceeds a certain **threshold**
- Capable to classify correctly **linearly separable** examples

Perceptron

Architecture with a **single neuron** (Rosenblatt, 1958)



Perceptron

How to learn parameters

- Define a **loss** function to be **minimized**
- Typically the **classification error** on the training set

$$E(\beta, \mathcal{D}) = \sum_{(x_i, y_i) \in \mathcal{D}} \ell(y_i, f(x_i))$$

- How can we **minimize** this error?

Perceptron



For the classic perceptron the **loss function** is defined as:

$$\ell(y_i, f(x_i)) = -y_i f(x_i)$$

Idea: **different signs** between target and prediction imply **wrong classifications**, so we should correct them

Perceptron

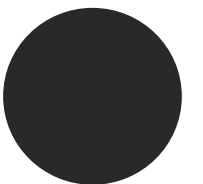
Gradient descent algorithm

- Compute the **gradient** of error function wrt parameters
- **Iterate** until gradient is approximately zero

$$\beta^i = \beta^{i-1} - \eta \nabla E(\beta, \mathcal{D})$$

**i IS THE
ITERATION**

- η is called the **learning rate**



Perceptron



The learning rate is a **crucial** parameter

- Too **small** implies slow **convergence**
- Too **high** causes **instability**
- There exist algorithms to **adapt it during training**

Guarantee to reach a **local minimum** of the error function

Perceptron



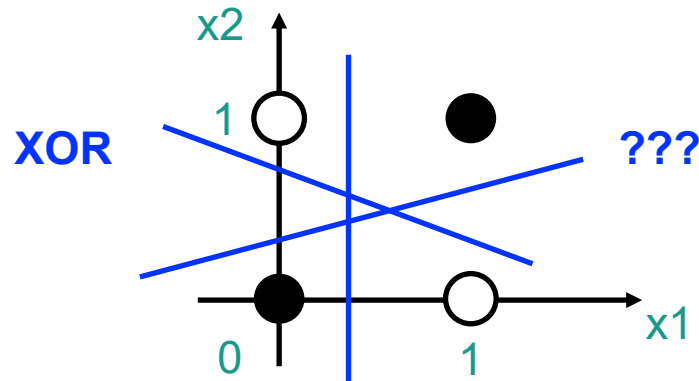
Batch vs. **stochastic** gradient descent

- In general, the gradient is obtained as the **sum** of gradients computed for all the **training examples**
- In principle, we could also update the vector of parameters **after each example** rather than after all the training set
- Update after each example is called **stochastic** update
- **Faster** training, which can help **avoiding** local minima
 - Helps avoiding positive and negative example cancelling each other
- **Mini-batch**

Perceptron

The perceptron finds a separating **hyperplane**

This is often **not enough**, because in many cases there exist **non-linear relationships** between input (features) and output (target)



Perceptron



Perceptron can also be used for **regression** problems

- Do **not** use threshold over the neuron output
- Exploit a **different loss function** (e.g., RMSE)

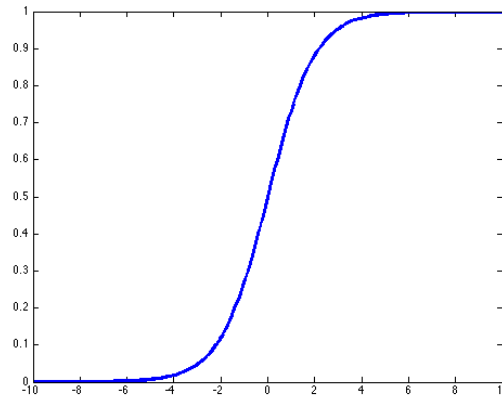
$$\ell(y_i, f(x_i)) = (y_i - f(x_i))^2$$

Logistic Regression

Modeling the probability of target being 1, given evidence

$$h(x) = p(y = 1|x) = \frac{1}{1+e^{-(\beta^T x + \beta_0)}} = \frac{e^{\beta^T x + \beta_0}}{1+e^{\beta^T x + \beta_0}}$$

The output lies between 0 and 1, as in the **sigmoid** function

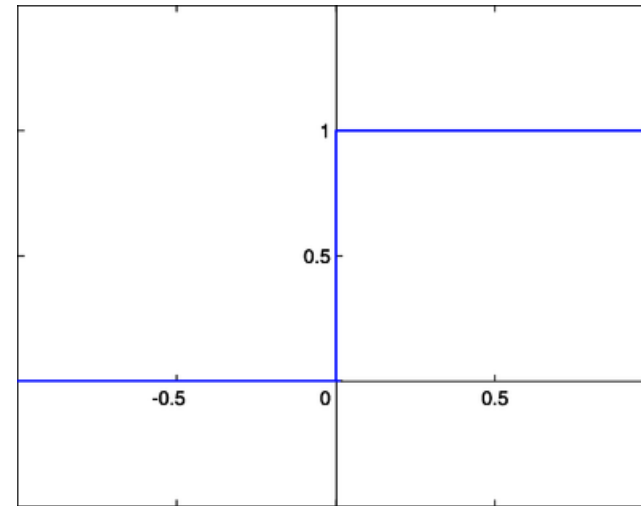
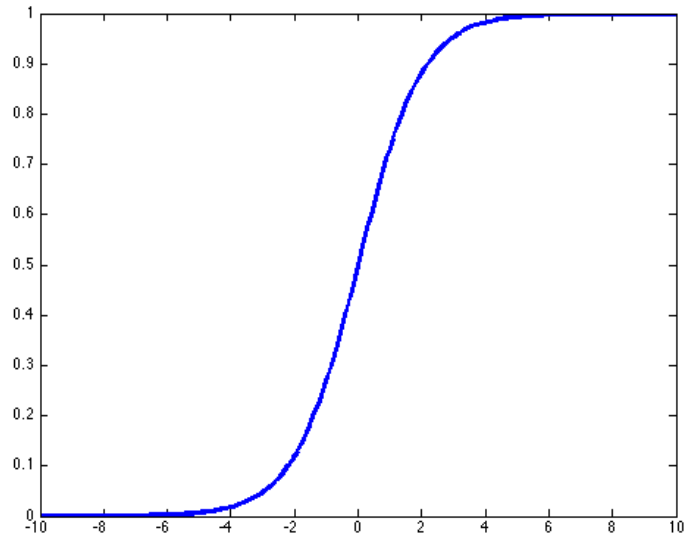


$$\sigma(t) = \frac{1}{1+e^{-t}}$$
$$t = \beta^T x + \beta_0$$

$$d/dt \sigma(t) = \sigma(t)(1-\sigma(t))$$

Logistic Regression

It is a sort of **smooth version** of the perceptron which exploits the **sigmoid** function in place of the **hard** threshold



Logistic Regression

In order to estimate the β coefficients, a typical approach is to **maximize** a so-called **likelihood function** β (note that the negative of the likelihood can be interpreted as as loss; maximize likelihood = minimize loss)

$$\ell(\beta, \mathcal{D}) = \sum_{(x_i, y_i) \in \mathcal{D}} y_i \log(h(x_i)) + (1 - y_i) \log(1 - h(x_i))$$

Basic idea: **fit** the β parameters so that the **output** of the logistic function is close to 1 for positive examples, and close to 0 for negative examples

Logistic Regression

Interpretability of the results

A **high-valued** parameter β means that the corresponding feature, if positive, is a strong indicator for **positive** class

A parameter β **close to zero** means that the corresponding feature is **irrelevant** for the classification

A **low-valued** parameter β means that the corresponding feature, if positive, is a strong indicator for **negative** class

Linear Regression



Linear regression with **least-squares** approach

Minimize the **Residual Sum of Squares** (RSS)

$$RSS = \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

where $\hat{y}_i$ is computed as

$$\hat{y}_i = \beta^T x = \sum_{i=1}^K \beta_i x_i$$

Linear Regression



In matrix form we are given N examples with K features: $Y \in \mathbb{R}^N, \beta \in \mathbb{R}^K, X \in \mathbb{R}^{N \times K}$

$$\min_{\beta} \|Y - \beta^T X\|^2$$

Linear vs. Non-Linear

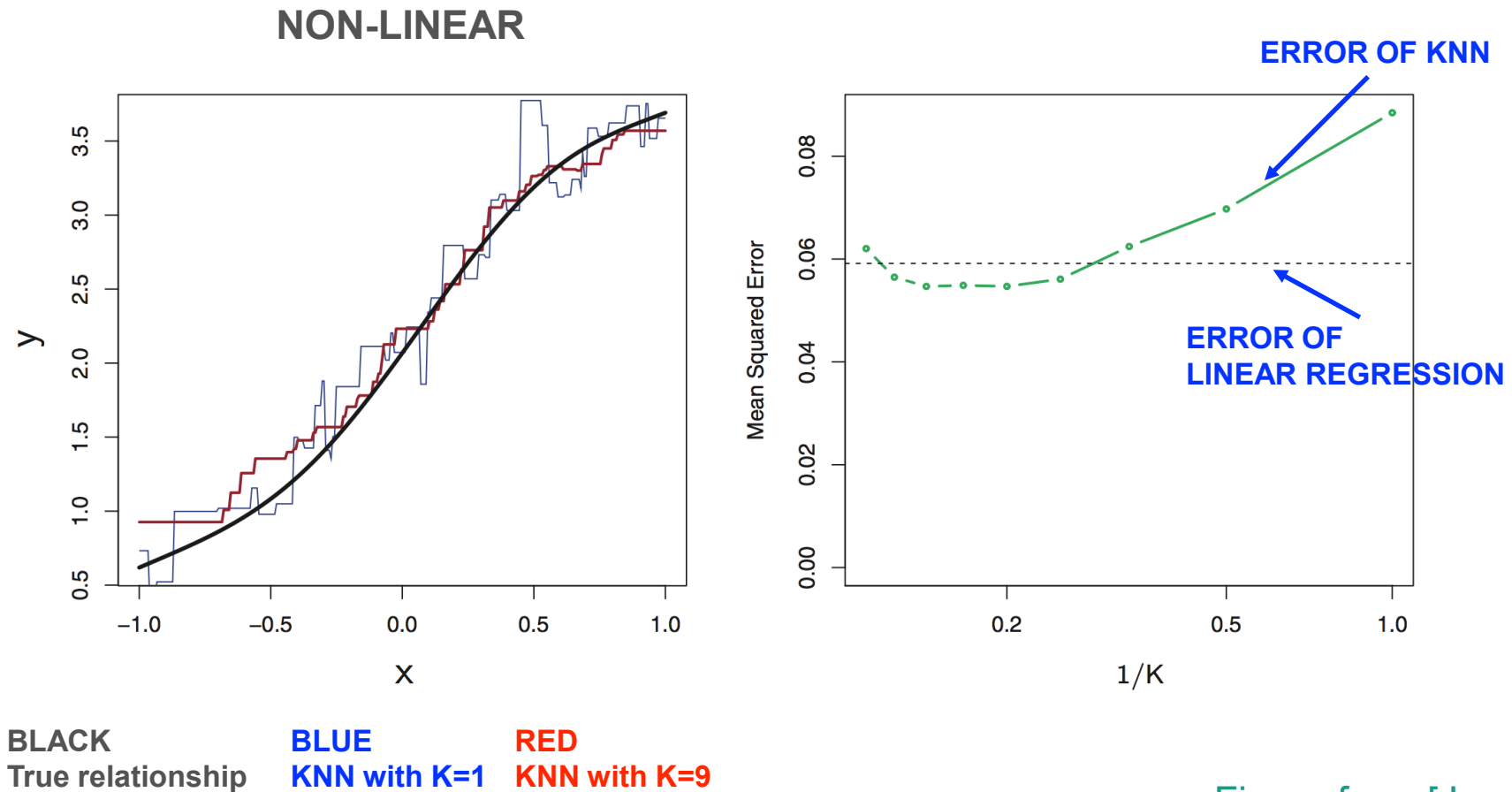
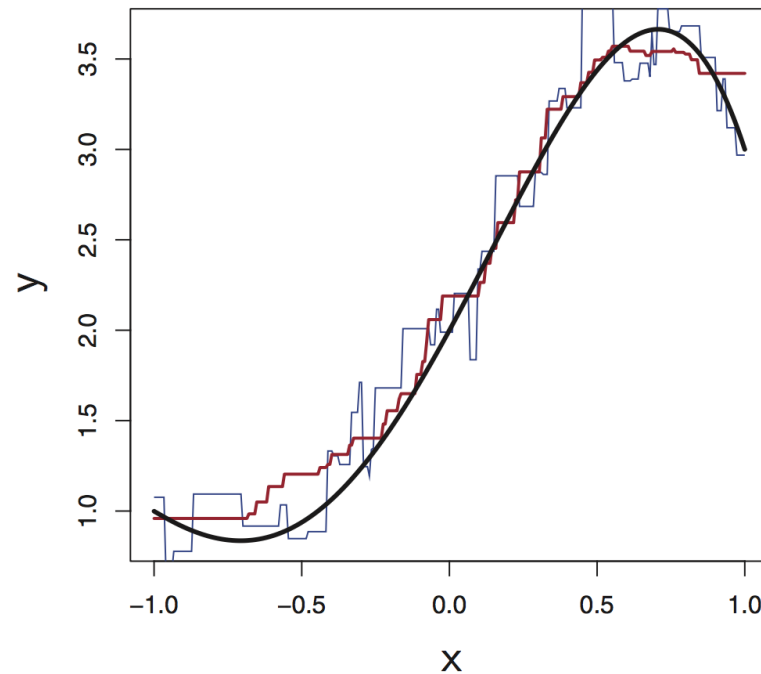


Figure from [James et al.]

Linear vs. Non-Linear

STRONGLY NON-LINEAR



BLACK
True relationship

BLUE
KNN with K=1

RED
KNN with K=9

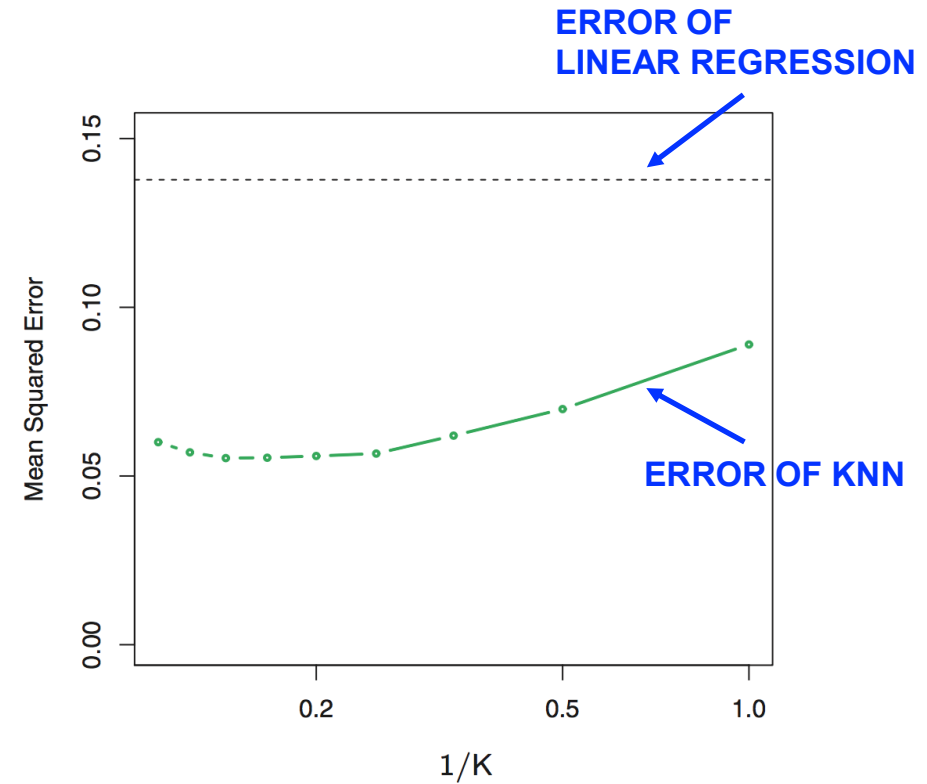


Figure from [James et al.]